



Instituto Tecnológico de Hermosillo

Ingeniería en Sistemas Computacionales

Programación Web — 8° Semestre — Grupo S6B

PROYECTO FINAL

GymMaster

Sistema de gestión de gimnasio y control de membresías

Integrantes del equipo

Duarte Bernal Sebastián

Ibarra Uriarte Sergio Osmar

Valdez Alfaro Cristopher Manuel

Profesor:

Jonathan Adrian Moroyoqui Ibarra

Fecha de entrega: 29 de mayo de 2026

1. Introducción

1.1 Descripción general del sistema

GymMaster es un sistema web cliente-servidor desarrollado para administrar la operación diaria de un gimnasio. El sistema cuenta con dos grandes perfiles de uso. El primero es el del Socio (Cliente), que puede registrarse, iniciar sesión, registrar su asistencia diaria al gimnasio, consultar los planes de membresía disponibles, contratar uno y acceder al detalle de los módulos(clases) que se encuentran activas en ese momento, con sus horarios e instructor asignado. El segundo perfil es el Administrativo (Admin / Recepcionista), que cuenta con un panel completo para administrar socios (CRUD), membresías (CRUD), instructores (CRUD), validar pagos pendientes, registrar asistencias de manera manual y configurar la visibilidad de los módulos que se muestran al cliente.

El sistema está construido sobre una arquitectura desacoplada: un Frontend Single Page Application en React que consume una API REST construida con ASP.NET Core. La persistencia de datos se realiza sobre SQL Server utilizando Entity Framework Core con el enfoque Code First, lo que permite versionar la base de datos mediante migraciones.

1.2 Problema que resuelve

Una gran parte de los gimnasios pequeños y medianos siguen administrando a sus socios y sus pagos en libretas, hojas de Excel sueltas o sistemas físicos como tarjetas de cartón. Esta forma de trabajar genera problemas concretos: pérdida de información cuando se mueven los registros, falta de visibilidad sobre quién ya pagó y quién no, dificultad para saber cuántos socios asistieron en un día y, en general, una experiencia poco profesional para el cliente final.

GymMaster digitaliza esta operación. Centraliza los datos de los socios, sus membresías y sus asistencias en una base de datos relacional; expone esa información mediante una API segura por capa de presentación; y entrega al cliente final una interfaz moderna donde puede registrar su asistencia en segundos y consultar los planes disponibles sin necesidad de pasar por recepción.

1.3 Objetivo general

Desarrollar un sistema web cliente-servidor que permita a un gimnasio administrar a sus socios, instructores, membresías y asistencias, separando claramente las funciones del usuario final (Cliente) y del personal administrativo (Admin / Recepcionista), aplicando las tecnologías vistas en el curso de Programación Web: ASP.NET Core Web API, Entity Framework Core con Code First, SQL Server, React con React Router y consumo de servicios REST.

2. Documentación técnica

2.1 Tecnologías utilizadas

Backend

- .NET 8 (TargetFramework: net8.0)
- ASP.NET Core Web API
- Entity Framework Core 8.x (paquete Microsoft.EntityFrameworkCore.SqlServer)
- Microsoft.EntityFrameworkCore.Tools 8.x (para migraciones desde la CLI dotnet ef)
- Swashbuckle.AspNetCore 6.6.2 (Swagger UI habilitado en modo desarrollo)
- CORS configurado para permitir el origen http://localhost:5173 (servidor de desarrollo de Vite)

Frontend

- React 19.2.6
- React DOM 19.2.6
- React Router DOM 7.15.1 (manejo de rutas SPA)
- Vite 8.0.12 (bundler y servidor de desarrollo)
- @vitejs/plugin-react 6.0.1
- ESLint 10.3.0 con plugins react-hooks y react-refresh

Base de datos

- SQL Server (probado con SQL Server Express, instancia local\\SQLEXPRESS01)
- Base de datos: GymMasterDB
- Esquema generado por completo desde Entity Framework Core (Code First). No se escribió ningún script SQL a mano.

2.2 Arquitectura

El proyecto está organizado en dos capas claramente separadas dentro de un mismo repositorio:

- GymMasterAPI/ contiene el Backend en ASP.NET Core.
- GymMasterAPI/vista/ contiene el Frontend en React (creado con Vite).

Estructura del Backend

- Controllers/ – Controladores que exponen los endpoints REST.
- Models/ – Entidades del dominio (Miembro, Instructor, Membresía, Asistencia).
- Data/ – ApplicationDbContext con la configuración de Entity Framework Core y el seed inicial.
- Dtos/ – Objetos de transferencia para separar la lectura, creación y actualización de cada entidad.
- Migrations/ – Migraciones generadas por EF Core (cuatro migraciones a la fecha).
- Program.cs – Punto de entrada que configura servicios (DbContext, Swagger, CORS) y el pipeline HTTP.

Estructura del Frontend

- src/components/ – Componentes de página (Home, Login, Register, InicioMiembro, GestionRecepcion, GestionInstructores, Membresias, ModuloDetalle) y sus hojas de estilo.
- App.jsx – Router que mapea las URLs públicas y privadas de la SPA.
- main.jsx – Bootstrap de React 19.

Comunicación Frontend ↔ Backend

La comunicación entre las dos capas se realiza mediante peticiones HTTP a la API REST. El frontend corre en `http://localhost:5173` y consume al backend en `http://localhost:5027` usando la API Fetch nativa de los navegadores.

El flujo típico es el siguiente: un componente de React lanza una petición fetch hacia un endpoint del controlador correspondiente; el controlador recibe la petición, valida el DTO de entrada, interactúa con la base de datos a través de Entity Framework Core y devuelve una respuesta JSON; el componente actualiza su estado con la respuesta y vuelve a renderizar la vista.

La autenticación se resuelve mediante una llamada a `POST /api/Miembros/login`: el backend valida el correo y la contraseña, y si son correctos devuelve el id, nombre y rol del usuario. El frontend guarda esta información en `localStorage` y la utiliza para decidir a qué vista redirigir y para mostrar u ocultar opciones del menú según el rol (Admin, Recepcionista o Cliente).

2.3 Base de datos

La base de datos cuenta con cuatro entidades principales generadas todas mediante migraciones de Entity Framework Core.

Entidad: Miembro

- Id (int, PK, autoincremental)
- Rol (string) – Valores: "Cliente", "Admin" o "Recepcionista". Por defecto Cliente.
- Nombre (string, requerido, máximo 100 caracteres)
- Email (string, requerido, validación de formato de correo)
- Password (string, requerido)
- FechaInscripcion (DateTime, se asigna al momento de crear el registro)
- EstaActivo (bool, por defecto true)
- Membresiald (int, opcional, llave foránea hacia Membresia)

Entidad: Membresia

- Id (int, PK)
- Nombre (string) – Ej. "Plan Diario", "Mensual + Módulo".
- Costo (decimal)
- DuracionDias (int)
- Descripcion (string)

Entidad: Instructor

- Id (int, PK)
- NombreCompleto (string)
- Especialidad (string) – Ej. "Boxeo", "Zumba y ritmos latinos".
- Turno (string) – "Matutino", "Vespertino" o "Matutino y Vespertino".
- EstaActivo (bool, por defecto true)

Entidad: Asistencia

- Id (int, PK)
- MiembroId (int, requerido, llave foránea hacia Miembro)
- FechaHora (DateTime, se asigna al momento de registrar la asistencia)

Entidad: Modulo

- Id (int, PK)
- Nombre (string, requerido) – Ej. "Boxeo", "Zumba".
- Descripcion (string)
- Badge (string, máximo 4 caracteres) – Sigla corta para la insignia visual. Ej. "BX", "ZB".
- Color (string) – Color hexadecimal con el que se tematiza la vista del módulo en el frontend. Ej. "#aa3bff".
- Activo (bool) – Controla la visibilidad del módulo para los clientes.
- InstructorId (int, opcional, llave foránea hacia Instructor)

Entidad: HorarioModulo

- Id (int, PK)
- Moduloid (int, requerido, llave foránea hacia Modulo)
- DiaSemana (string) – Lunes, Martes, Miércoles, Jueves, Viernes, Sábado o Domingo.
- HoraInicio (string, formato 24 horas: "07:00")
- HoraFin (string, formato 24 horas: "08:30")
- Nivel (string) – "Todos los niveles", "Principiante", "Intermedio" o "Avanzado".

Relaciones entre tablas

- • Membresia 1 — N Miembros (un miembro tiene cero o una membresía vigente; cada membresía puede estar asignada a muchos miembros).
- • Miembro 1 — N Asistencias (un mismo socio puede registrar muchas asistencias a lo largo del tiempo).
- • Instructor 1 — N Modulos (cada módulo puede tener un instructor asignado; un mismo instructor puede impartir varios módulos).
- • Modulo 1 — N HorariosModulo, con borrado en cascada (al eliminar un módulo se eliminan automáticamente todos sus horarios).

Migraciones

- 20260525232455_InitialCreate – Crea las tablas Miembros, Membresías e Instructores.
- 20260526035444_AgregarAsistencias – Agrega la tabla Asistencias con su llave foránea hacia Miembros.
- 20260527052907_AgregarRolesDB – Agrega la columna Rol a la tabla Miembros.
- 20260527194144_AgregarInstructores – Seed inicial con 6 membresías, 2 instructores y 2 cuentas administrativas (Admin y Recepcionista).
- AgregarModulos – Crea las tablas Modulos y HorariosModulo con sus relaciones, y aplica el seed inicial de 2 módulos (Boxeo y Zumba) con 8 horarios distribuidos.

Cuentas creadas por el seed

- admin@gym.com / admin123 — Rol: Admin
- recepcion@gym.com / recepcion123 — Rol: Recepcionista

2.4 API – Endpoints desarrollados

A continuación se listan los endpoints expuestos por el backend. Todos están disponibles en Swagger UI en la ruta /swagger cuando el proyecto corre en modo desarrollo.

Miembros (Socios)

Método	Ruta	Descripción
GET	/api/Miembros	Obtener el listado de todos los socios (incluye su membresía).
GET	/api/Miembros/{id}	Obtener un socio por su Id.
POST	/api/Miembros	Registrar un nuevo socio.
POST	/api/Miembros/login	Iniciar sesión. Devuelve id, nombre, email y rol.
PUT	/api/Miembros/{id}	Actualizar los datos de un socio existente.
DELETE	/api/Miembros/{id}	Eliminar un socio del sistema.

Membresías

Método	Ruta	Descripción
GET	/api/Membresias	Obtener todos los planes de membresía.
GET	/api/Membresias/{id}	Obtener una membresía por su Id.
POST	/api/Membresias	Crear un nuevo plan de membresía.
PUT	/api/Membresias/{id}	Actualizar un plan existente.
DELETE	/api/Membresias/{id}	Eliminar un plan.

Instructores

Método	Ruta	Descripción
GET	/api/Instructores	Obtener todos los instructores.
GET	/api/Instructores/{id}	Obtener un instructor por su Id.
POST	/api/Instructores	Registrar un nuevo instructor.
PUT	/api/Instructores/{id}	Actualizar los datos de un instructor.
DELETE	/api/Instructores/{id}	Eliminar un instructor.

Módulos (Clases)

Método	Ruta	Descripción
GET	/api/Modulos	Obtener todos los módulos con su instructor asignado y la lista completa de horarios.
GET	/api/Modulos/{id}	Obtener un módulo por su Id.
POST	/api/Modulos	Crear un nuevo módulo enviando también su lista de horarios en una sola petición.
PUT	/api/Modulos/{id}	Actualizar un módulo y reemplazar por completo la lista de horarios asociados.
DELETE	/api/Modulos/{id}	Eliminar un módulo (sus horarios se eliminan en cascada).

Asistencias

Método	Ruta	Descripción
POST	/api/Asistencias/regar	Registrar la asistencia de un socio. Recibe el Id del socio y guarda la fecha y hora actuales.

2.5 Vistas del Frontend

El frontend cuenta con ocho rutas distintas que se distribuyen entre los dos perfiles de usuario.

- / – Página de bienvenida (Home) con botones de Iniciar Sesión y Registrarse.
- /login – Pantalla de inicio de sesión. Redirige a /gestion-recepcion o /inicio-miembro según el rol.
- /registro – Formulario de alta para nuevos socios.
- /inicio-miembro – Pantalla principal del socio. Pide confirmar asistencia (check-in) y luego muestra los módulos activos configurados por la administración.
- /modulo/:tipo – Detalle del módulo (Boxeo o Zumba). Muestra horarios, instructor asignado (consumido desde la API) y permite registrar asistencia a la clase.
- /membresias – Catálogo de planes de membresía. Permite simular la contratación de un plan.
- /gestion-recepcion – Panel administrativo. Cuenta con cuatro pestañas: Socios, Clases, Membresías y Staff.
- /gestion-instructores – Pantalla CRUD dedicada al staff de instructores.

3. Distribución del trabajo

La distribución se basa en el historial real de commits del repositorio (rama develop). Cada integrante asumió un rol predominante pero las decisiones de arquitectura y los ajustes de integración se tomaron en equipo.

Ibarra Uriarte Sergio Osmar

Backend

- Implementación del endpoint POST /api/Asistencias/registrar con validación de existencia del socio.
- Implementación del CRUD completo de Membresías sobre el controlador MembresiasController.
- Diseño e implementación del sistema de roles en el modelo Miembro (Cliente / Recepcionista / Admin) y la migración correspondiente.

Frontend

- Diseño visual del Login y de la pantalla de bienvenida del socio.
- Implementación de la lógica de check-in del socio y conexión con el endpoint de Asistencias.
- Diseño del panel administrativo unificado en GestionRecepcion: pestañas dinámicas de Socios, Clases, Membresías y Staff.
- Implementación de los módulos dinámicos para el cliente: la administración decide qué clases se muestran en /inicio-miembro.
- Validación de pagos / membresías y corrección del bypass de login para cuentas inactivas.

Valdez Alfaro Cristopher Manuel

Frontend

- Implementación inicial de las pantallas Home, Login y Register.
- Diseño e implementación de las vistas de módulos (Boxeo y Zumba) en ModuloDetalle.
- Diseño del catálogo de planes en /membresías y de los flujos de contratación.
- Integración de las pantallas administrativas: gestión de socios, clases y staff.
- Estandarización del navbar del panel administrativo y corrección del bug de cierre de sesión.
- Corrección del buscador en la pestaña de clases.

Sebastián Duarte Bernal

Backend

- Inicialización del backend en ASP.NET Core: configuración del proyecto, estructura de carpetas y connection string.
- Creación de los modelos: Miembro, Membresia, Instructor, Asistencia, Modulo y HorarioModulo.
- Configuración del ApplicationDbContext con todos los DbSet, las relaciones entre entidades y la política de borrado en cascada para los horarios.
- Diseño y creación de los DTOs para separar la lectura, creación y actualización (RegistrarMiembroDto, ActualizarMiembroDto, MiembroDto, CrearMembresiaDto, MembresiaDto, CrearInstructorDto, InstructorDto, LoginDto, ModuloDto, CrearModuloDto, HorarioModuloDto, CrearHorarioModuloDto).
- Generación de todas las migraciones de Entity Framework Core (InitialCreate, AgregarAsistencias, AgregarRolesDB, AgregarInstructores y AgregarModulos) y configuración del seed inicial completo (6 membresías, 2 instructores, 2 módulos con 8 horarios y 2 cuentas administrativas).
- Implementación del CRUD completo de Instructores sobre el controlador InstructoresController.
- Implementación del CRUD completo de Módulos (clases) sobre el controlador ModulosController, incluyendo el manejo anidado de horarios al crear o actualizar un módulo.

Frontend

- Implementación de la vista CRUD GestionInstructores.jsx para administrar al staff.
- Refactor de ModuloDetalle.jsx e InicioMiembro.jsx para que las clases y sus horarios se carguen desde el endpoint /api/Modulos en lugar de listas hardcodeadas en el frontend.
- Rediseño del modal de configuración de clases en GestionRecepcion.jsx: selector estructurado de día, hora de inicio, hora de fin y nivel, con soporte para múltiples horarios por clase.

4. Retroalimentación del proyecto

4.1 Ibarra Uriarte Sergio Osmar

¿Qué fue lo más difícil de desarrollar y cómo lo resolvieron?

Se me complicó mucho las validaciones de los login dependiendo del rol ya que no encontraba cómo resolverlo, tome ideas de proyectos hechos anteriormente para poder resolver ese problema.

¿Qué herramientas o recursos externos consultaron?

IA (Gemini), y videos en youtube para la resolución de problemas: como conectar base de datos ya que en ocasiones tenía problemas de conexión cuando hacía el pull.

¿Qué cambiarían o mejorarían del proyecto si tuvieran más tiempo?

Cambiaría el sistema de pagos, que te permita decidir si seguir pagando con código QR o con tarjeta, además me gustaría implementar un apartado de proveedores, ya sea para vender ropa o suplementos dentro del gimnasio.

¿Qué aprendieron que no sabían antes de iniciar el proyecto?

Conocí cómo se manejan los endpoints al hacer cualquier acción dentro de la página, también aprendí que es mejor clonar o dependiendo de la circunstancias del proyecto, hacer pull al mismo, para no tener problemas al momento de ejecutar la base de datos o la página.

4.2 Valdez Alfaro Cristopher Manuel

¿Qué fue lo más difícil de desarrollar y cómo lo resolvieron?

Lo más difícil fueron las migraciones, ya que al inicio generaban muchos errores y, al consultar en SQL Server, no se mostraba realmente creada la base de datos. Pero, mediante los componentes que vimos en clase como la importación de Entity Framework y el acomodo correcto de los atributos en el ApplicationDbContext se logro ejecutar los comandos en la terminal y sincronizar todo correctamente.

¿Qué herramientas o recursos externos consultaron?

Quando necesitaba arreglar un problema hacía búsquedas simples dentro de la web, o utilicé la herramienta gemini, y también hice búsquedas de vídeos en youtube aunque estos realmente no fueron de gran ayuda.

¿Qué cambiarían o mejorarían del proyecto si tuvieran más tiempo?

La estructura realmente no me convence del todo, pero lo que cambiaría sería la implementación de los métodos de pago mediante Stripe, ya que esta tecnología le hubiera dado un entorno más profesional al proyecto para poder realizar los pagos en línea y a su vez habilitar más funciones dentro del apartado del admin.

¿Qué aprendieron que no sabían antes de iniciar el proyecto?

Pues aunque suene algo simple yo no había implementado nunca la creación de los códigos QR, ya que tuve que buscar una api de un servicio que me pudiera generar, y para no provocar que se retrasara mucho el proyecto al agregar los métodos de pago implemente que se mandará el id del cliente hacia el

servicio de qr para que así lo pudiera mostrar dentro de la página, lo cual me pareció algo muy interesante ya que nunca había trabajado de esa forma.

4.3 Duarte Bernal Sebastian

¿Qué fue lo más difícil de desarrollar y cómo lo resolvieron?

la implementación de los dtos, ya que no entendía bien cómo funcionaban

¿Qué herramientas o recursos externos consultaron?

varios videos de youtube, y con el uso de gemini para resolver ciertas dudas o mejoras del front

¿Qué cambiarían o mejorarían del proyecto si tuvieran más tiempo?

Le pondría mejoras para que el cliente pueda pagar en línea, y no tenga que ir a caja, esto si fuera un sistema real, que se le vendió a un gimnasio, y también agregaría una sección de tienda, con artículos de gym

¿Qué aprendieron que no sabían antes de iniciar el proyecto?

Aprendi mas sobre como hacer las migraciones de la base de datos, porque en clase no la pude hacer funcionar, y tambien aprendi mas del uso del github

5. Retroalimentación del curso

Cada integrante responde de manera individual (puede ser anónimo si así lo prefiere).

5.1 Integrante 1

¿Qué temas del curso les fueron más útiles para el proyecto?

Principalmente, como crear la página desde react, el cómo se manejan los endpoints y la migración a la base de datos.

¿Qué temas sintieron que necesitaban más práctica o explicación?

Todos tuvieron un ritmo adecuado para entenderlos y estudiarlos, además de las prácticas realizadas.

¿Qué le agregarían o quitarían al curso?

Sinceramente creo que todo estuvo al alcance.

¿Cómo calificarían el curso del 1 al 10 y por qué?

10, porque la actitud, la paciencia y la forma de dar la clase que tuvo el maestro me agradó bastante, además de que la clase en sí, fue de mis materias favoritas ya que es a lo que me gustaría enfocarme.

Comentarios adicionales para el profesor

Muchas gracias por su manera de impartir la clase, se nota que es algo a lo que le ha dedicado tiempo y que lo disfruta.

5.2 Integrante 2

¿Qué temas del curso les fueron más útiles para el proyecto?

El tema que se vio para aplicar correctamente los dtos, y también el cómo realizar las migraciones correctamente

¿Qué temas sintieron que necesitaban más práctica o explicación?

Realmente siento que todo el curso se cubrió correctamente, ya que algunos temas hasta abarcaron varias clases para poder entenderlos correctamente.

¿Qué le agregarían o quitarían al curso?

Considero que tal como esta se lleva correctamente y logra cumplir en tiempo y forma todo el semestre.

¿Cómo calificarían el curso del 1 al 10 y por qué?

La calificación que le daría sería un 10/10, ya que siento que se lleva de forma correcta, al inicio del curso realizando las exposiciones se logró dar como las bases para que nosotros entiéramos lo principal dentro de la web, y cada clase se fue viendo diferentes componentes y temas importantes para entender toda la estructura.

Comentarios adicionales para el profesor

Pues que todo bien, considero que es buena onda, y realmente conoce muy bien de todos los temas del curso, y está siempre muy pendiente para darnos soluciones a problemas que nosotros tengamos.

5.3 Integrante 3

¿Qué temas del curso les fueron más útiles para el proyecto?

Todos los temas vistos en clase fueron útiles para el desarrollo del proyecto, desde cómo levantar una página con react, y utilizar el .net para la api

¿Qué temas sintieron que necesitaban más práctica o explicación?

todos estuvieron bien a mi parecer, fueron prácticas bien realizadas y con buen contenido

¿Qué le agregarían o quitarían al curso?

no le movería nada, muy buen curso

¿Cómo calificarían el curso del 1 al 10 y por qué?

10, porque sentí que aprendí más en esta clase que en cualquier otra, y me sirvió para mis demás clases

Comentarios adicionales para el profesor

Buena bola profe, se la rifo, gracias

6. Conclusiones

6.1 Conclusión general del equipo

Al cierre del proyecto, GymMaster cumple con el objetivo planteado desde el inicio: ofrecer un sistema funcional de gestión de gimnasio con dos perfiles de usuario claramente diferenciados, persistencia real en SQL Server, una API REST documentada con Swagger y una interfaz de usuario construida íntegramente en React. El sistema cubre los requisitos solicitados por la rúbrica: tres CRUDs completos (Miembros, Membresías e Instructores), dos módulos funcionales (Boxeo y Zumba), siete vistas distintas en el frontend y la base de datos generada por completo a través de migraciones.

6.2 ¿Logramos el objetivo planteado?

Sí. El objetivo inicial era construir un sistema cliente-servidor que permitiera administrar socios, instructores, membresías y asistencias separando claramente los roles. Se logró tanto la parte administrativa (panel de Recepción / Admin con CRUD completo y configuración de visibilidad de módulos) como la parte del cliente final (registro, check-in, vista de módulos, contratación de planes). Adicionalmente, el equipo logró integrar funciones que no estaban en el plan inicial: el sistema de roles con redirección automática según el perfil, la configuración dinámica de módulos visibles, la validación de pagos pendientes y el seed inicial con cuentas administrativas pre-creadas.

6.3 Reflexión final sobre el trabajo en equipo

El trabajo en equipo se sostuvo gracias a una división clara de responsabilidades desde el principio: una persona se enfocó en sentar las bases del backend (modelos, DbContext, DTOs y migraciones), otra en el flujo del cliente final y el panel administrativo, y otra en las pantallas de inicio y los módulos. La rama develop sirvió como punto único de integración y permitió que cada integrante avanzara en paralelo sin pisarse el trabajo. Los puntos donde más se notó el valor de trabajar en equipo fueron la integración del panel administrativo (varios CRUDs viviendo en la misma pantalla con pestañas) y los ajustes de UX, donde cada quien aportó ideas sobre cómo simplificar los flujos. Quedó claro para todos que mantener un repositorio limpio, hacer commits descriptivos y comunicarse antes de tocar el código compartido fueron los factores que más impacto tuvieron en la velocidad final del proyecto.